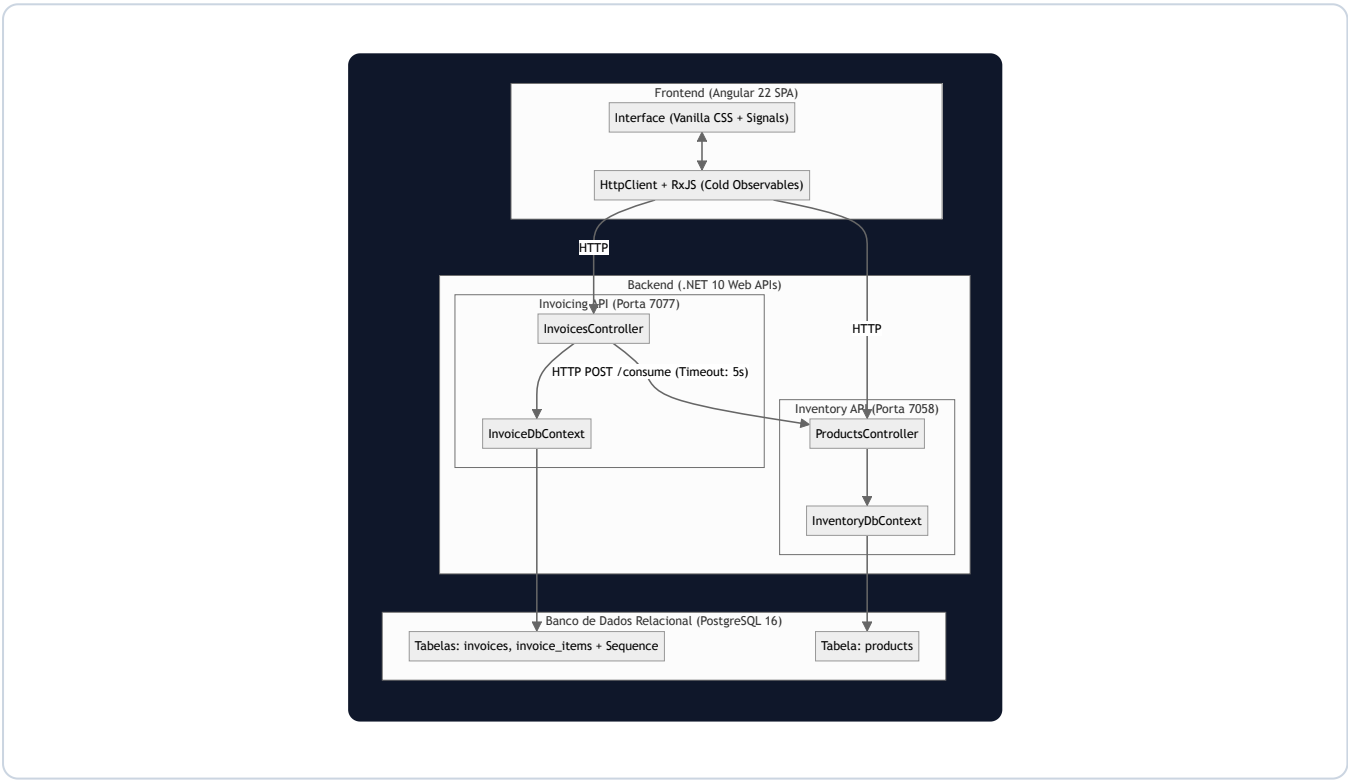


Detalhamento Técnico da Solução: Sistema de Emissão de Notas Fiscais e Controle de Estoque

Candidato: João Marcos Silva	Empresa: Korp Informática Ltda
Vaga: Estágio de Desenvolvimento C# ou Go + Angular	Repositório: github.com/joao-m-csilva/Korp_Testes_JoaoMarcosSilva

1. Visão Geral da Arquitetura e Decisões de Design

A solução foi projetada sob o paradigma de **Arquitetura de Microsserviços**, desacoplando os domínios de negócio em dois serviços de backend independentes e uma aplicação frontend em **Single Page Application (SPA)**, com persistência física em **PostgreSQL** e orquestração completa em **Docker**.



Principais Decisões Arquiteturais:

- Desacoplamento de Domínio:** O serviço de Faturamento (**Invoicing API**) não acessa diretamente as tabelas de estoque no banco de dados. Qualquer operação de baixa ou consulta de saldo é mediada exclusivamente por contratos de API REST via protocolo HTTP.
- Isolamento de Persistência:** Embora ambos os serviços compartilhem o mesmo servidor PostgreSQL orquestrado no Docker, cada microsserviço possui seu próprio **DbContext** isolado, garantindo governança estrita sobre seus próprios dados e migrações.
- Resiliência e Tolerância a Falhas:** A comunicação entre microsserviços foi blindada contra lentidão e quedas de serviço através de timeouts configurados e captura de exceções de rede com retorno de status HTTP apropriado.

2. Frontend (Angular 22)

2.1. Frameworks e Bibliotecas Utilizadas

- **Angular 22:** Utilização dos recursos mais modernos do ecossistema Angular:
 - **Standalone Components:** Eliminação do overhead de `NgModule`, tornando os componentes modulares e de carregamento eficiente via Lazy Loading no roteador.
 - **Angular Signals (`signal` , `computed`):** Primitiva reativa moderna para gerenciamento de estado fino e atualização pontual de nós do DOM sem necessidade de re-renderização total da árvore de componentes.
 - **Novo Control Flow (`@if` , `@for`):** Sintaxe nativa do Angular que substitui as antigas diretivas `*ngIf` e `*ngFor`, com melhoria de performance e legibilidade.
- **RxJS (~7.8.0):** Biblioteca de programação reativa assíncrona integrada ao `HttpClient`.
- **@angular/forms (FormsModule):** Utilizado para two-way data binding em formulários reativos simples nos modais de cadastro.
- **@angular/router:** Gerenciamento de rotas e navegação SPA.
- **Vitest (~4.0.8) & JSDOM (~28.0.0):** Suite de testes unitários automatizados moderna e de alta performance.

2.2. Decisão sobre Bibliotecas Visuais (Componentes de UI)

Optou-se por **não utilizar bibliotecas de terceiros pesadas** (como PrimeNG, Angular Material ou Bootstrap). Desenvolveu-se um design system próprio em **Vanilla CSS autoral** com layout responsivo, variáveis CSS, modais acessíveis, alertas visuais semânticos e ícones SVG inline. Essa decisão garantiu um bundle extremamente leve e total controle sobre os estilos.

2.3. Ciclos de Vida do Angular Utilizados

O ciclo de vida `ngOnInit` foi implementado nos componentes `Home`, `Produtos` e `NotasFiscais`. Como a aplicação opera como SPA, ao navegar pelas rotas, o componente é instanciado e o `ngOnInit` atua como o ponto de entrada seguro para disparar a carga inicial de dados via serviços HTTP.

2.4. Uso da Biblioteca RxJS e Padrão de Observables

- **Cold Observables (Unicast 1:1):** As chamadas geradas pelo `HttpClient` operam como Cold Observables. A requisição HTTP só é efetivamente disparada no momento da subscrição (`.subscribe()`), recebendo exatamente uma resposta e completando o ciclo (`complete`) automaticamente.
- **Chamadas Desacopladas para Tolerância a Falhas:** No painel inicial (`Home`) e na tela de `NotasFiscais`, as chamadas para a API de Estoque e para a API de Faturamento foram estruturadas com subscrições independentes. Se o serviço de Estoque estiver fora do ar, o sistema não quebra por completo: ele carrega os dados de Faturamento normalmente e exibe um alerta pontual informando a indisponibilidade do Estoque.

3. Backend (C# / .NET 10)

3.1. Frameworks e Gerenciamento de Dependências

- **Plataforma:** .NET 10 / ASP.NET Core Web API.
- **ORM:** Entity Framework Core 10 (`Microsoft.EntityFrameworkCore` , `Npgsql.EntityFrameworkCore.PostgreSQL`).
- **Documentação de API:** `Microsoft.AspNetCore.OpenApi` (OpenAPI/Swagger).
- **Gerenciamento de Dependências:** Gerenciado nativamente pelo **NuGet** através dos arquivos `.csproj` .

3.2. Estrutura dos Microserviços

1. **Korp.Inventory.API (Porta 7058):**
 - `GET /api/products` : Listagem de produtos.
 - `GET /api/products/{id}` : Consulta individual.
 - `POST /api/products` : Cadastro com validações estritas de domínio.
 - `POST /api/products/consume` : Endpoint transacional de baixa em lote de estoque.
2. **Korp.Invoicing.API (Porta 7077):**
 - `GET /api/invoices` : Listagem de notas com itens relacionados.
 - `GET /api/invoices/{id}` : Consulta detalhada.
 - `POST /api/invoices` : Emissão de nova nota fiscal (situação inicial: "Aberta").
 - `POST /api/invoices/{id}/print` : Orquestração do fechamento da nota e consumo de estoque no serviço de inventário.

4. Uso do LINQ em C#

4.1. Consultas e Relacionamentos no Banco de Dados (LINQ to Entities / EF Core)

O Entity Framework Core traduz as expressões LINQ em queries SQL nativas e otimizadas para o PostgreSQL:

- **Carregamento de Relacionamentos:** `.Include(i => i.Items)` realiza o `LEFT JOIN` entre as tabelas `invoices` e `invoice_items` .
- **Ordenação Cronológica:** `.OrderByDescending(i => i.Date)` garante a exibição das notas mais recentes primeiro.
- **Buscas Precisas:** `.FirstOrDefaultAsync(p => p.Id == id)` e `.FirstAsync( ... )` .
- **Otimização de Leitura:** `.AsNoTracking()` para consultas somente leitura, reduzindo o custo de rastreamento de entidades no DbContext.

4.2. Lógica de Negócio, Validações e Projeções em Memória (LINQ to Objects)

- **Cálculo do Total da Nota (`Sum`):**

```
invoice.Total_Value = invoice.Items.Sum(item => item.Quantity * item.UnitPrice);
```

- **Projeção e Transformação de DTOs (`Select`)**: Mapeia a entidade rica de itens de nota fiscal para o payload enxuto exigido pela API de Estoque:

```
var request = new ConsumeStockDto
{
    Items = invoice.Items
        .Select(item => new ConsumeStockItemDto
        {
            ProductId = item.ProductId,
            Quantity = item.Quantity
        })
        .ToList()
};
```

- **Validações de Regra de Negócio (`Any` e `All`)**:

```
if (invoice.Items.Any(i => i.Quantity <= 0)) { return BadRequest("Quantidade inválida."); }
if (!product.Code.All(char.IsDigit)) { return BadRequest("Código deve ser numérico."); }
```

5. Tratamento de Erros, Exceções e Resiliência

5.1. Resiliência na Comunicação entre Microserviços (Cenário de Falha)

Para cumprir o requisito obrigatório de recuperação e tratamento de falhas:

- A `Invoicing API` registra o `HttpClient` com **Timeout de 5 segundos**.
- **Interceptação de Falha de Rede**: No método de impressão, a chamada externa é protegida por `try/catch` capturando `HttpRequestException` e retornando **503 Service Unavailable**:

```
try {
    var response = await ConsumeInventoryAsync(invoice);
    if (!response.IsSuccessStatusCode) {
        var error = await response.Content.ReadAsStringAsync();
        return new ContentResult { Content = error, ContentType = "application/json", StatusCode = 503 };
    }
} catch (HttpRequestException) {
    return StatusCode(503, new {
        message = "Serviço de inventário indisponível no momento. Não foi possível baixar o e"
    });
}
```

- **Garantia de Integridade**: Se o estoque estiver indisponível, a nota **não** tem seu status alterado para "Fechada", evitando inconsistência de dados.

5.2. Tratamento de Erros de Banco de Dados (Chave Duplicada)

No cadastro de produtos, colisões de código único geram `DbUpdateException`. O controller intercepta a `PostgresException` de violação de chave única (código 23505) e responde com status **409 Conflict**.

5.3. Validações de Domínio e Entrada (400 Bad Request)

Validação de descrições com menos de 10 caracteres, saldos menores ou iguais a zero, regras para unidades inteiras (UN, PC, CX) e bloqueio de impressão para notas com status diferente de "Aberta".

5.4. Middleware Global de Exceções (500 Internal Server Error)

Configurado via `app.UseExceptionHandler()` em ambas as APIs para interceptar qualquer erro inesperado, devolvendo JSON amigável e evitando vazamento de stack traces em produção.

6. Persistência e Modelagem no PostgreSQL

- **PostgreSQL 16 Alpine:** Executando em container Docker com volume persistente (`postgres_data`).
- **Migrações Automáticas:** Execução de `db.Database.Migrate()` na inicialização dos containers, aplicando o schema relacional e inserindo uma carga inicial de 50 produtos industriais (Seed Data).
- **Numeração Sequencial Garantida no Banco:** Controlada diretamente por uma **Sequence nativa do PostgreSQL** (`invoice_number_seq`), garantindo atomicidade e incremento ordenado sem falhas de concorrência.

7. Quadro Resumo de Tecnologias

Camada	Tecnologia / Recurso	Finalidade
Frontend	Angular 22 (Standalone + Signals)	Interface reativa, moderna e de alta performance
Comunicação Front	RxJS (Cold Observables)	Requisições assíncronas desacopladas via <code>HttpClient</code>
Estilização	Vanilla CSS Autoral	Design system responsivo sem bibliotecas pesadas de terceiros
Testes Front	Vitest + JSDOM	Testes unitários automatizados
Backend	.NET 10 / ASP.NET Core	Microserviços REST rápidos e fortemente tipados
ORM / Banco	EF Core 10 + Npgsql	Mapeamento relacional e migrations no PostgreSQL
Consultas / Lógica	LINQ (C#)	Consultas otimizadas no banco e manipulação de DTOs em memória
Banco de Dados	PostgreSQL 16 Alpine	Persistência relacional física com Sequence nativa
Orquestração	Docker & Docker Compose	Execução padronizada e reproduzível de toda a infraestrutura

8. Como Executar o Projeto

```
# 1. Executar o Backend e Banco de Dados
docker compose up --build -d
```

```
# 2. Executar o Frontend
cd frontend
npm install
npm start
```

```
# 3. Executar os Testes do Frontend
cd frontend
npm test -- --watch=false
```